

```

        Console.WriteLine(op2(1, 1));
    }
}

```

Delegate objects can reference not just one method, but multiple methods. These are referred to as multicast delegates. Note that if a delegate specifies a return value/out parameters that the result of the call is the result of the last method that is invoked.;

```

public delegate void PrintStuff();
public static class PrintStuffTest
{
    public static void PrintHello() { Console.
WriteLine("Hello"); }
    public static void PrintWorld() { Console.
WriteLine("World"); }
    public static void PrintBang() { Console.WriteLine("!");
}

    public static void Test()
    {
        PrintStuff p = PrintHello;
        p += PrintWorld;
        p += PrintBang;
        p += PrintBang;
        p += PrintBang;
        p();
    }
}

```

Delegates that contain only one method derive from `System.Delegate`. Likewise, delegates that contain multiple methods derive from `System.MulticastDelegate`. Delgates can also be made generic just like a concrete method.

```

public static class GenericDelegateTest
{
    delegate T Process<T>(T t);
    public static int Inc(int i) { return i+1; }
    public static void Test()
    {
        Process<int> p = Inc;
        Console.WriteLine(p(0));
    }
}

```